University of Exeter

The C Family
ECM2433

Basic C

Richard Everson

Hello World!

University of Exeter

```
/* My sample hello world program */
#include <stdio.h>

int main()
{
   printf("Hello world!\n");
   return 0;
}
```

comment

compiler directive

main function

start block/scope

output some text

finish with succes

end block/scope

Recap…

- Comments are delimited by: `/* … */`
- In C99 `// comment to the end of the line` is permitted
- `#include <stdio.h>` declares the `printf()` function
- Every program has exactly one entry point, `main()`
- Function bodies and block are delimited by `{ … }`
- Statements are terminated with semicolons `;`
- Literal strings are delimited by double quotes: `"String"`
- The escape code `\n` is a single character meaning newline
- `printf()` prints a string and other variables
- The `main()` function returns an integer to the operating system

- Indentation is unimportant to the compiler, but vital for humans

Auto variables

University of Exeter

```
int main ()
{
   /* declare some variables */
   int    anInteger;              /* an integer */
   float  aDecimal;               /* a floating point number */
   double someBigDecimal;         /* a big floating point number */
   char   oneCharacter   = 'X';   /* a single character, initialised */
   int    anotherInteger = 4;     /* another integer, initialised */
   int    an_old_school_variable_name;

   /* do some stuff */

   return 0;
}
```

- C is statically typed: Variables must be declared before use
- Space for variables is allocated on the stack
- Known as ‘automatic’ or ‘auto’ variables

Base data types

Examples of machine-specific sizes, but “mostly” universal

Data type	Size	Range
unsigned char	1 byte	0 to 255
char	1 byte	-128 to 127
unsigned short int	2 bytes	0 to 65,535
short int	2 bytes	-32,768 to 32,767
int	4 bytes	-2,147,483,648 to 2,147,483,647
long int	8 bytes	$\pm 9 \times 10^{18}$

The only guarantees are:
char = exactly 1 byte
sizeof short int ≥ sizeof char
sizeof int ≥ sizeof short int
sizeof long int ≥ sizeof int
etc

Type	Size	smallest value	largest value	precision
float	4 bytes	$\pm 3.4 \times 10^{-38}$	$\pm 3.4 \times 10^{38}$	6 digits
double	8 bytes	$\pm 1.07 \times 10^{-308}$	$\pm 1.07 \times 10^{308}$	15 digits
long double	16 bytes	$\pm 3.4 \times 10^{-4932}$	$\pm 3.4 \times 10^{4932}$	18 digits

Data types: size and range

```
#include <stdio.h>
#include <limits.h>
int main ()
{
    char letter = 'E';
    printf("Min and max char: %d, %d\n", CHAR_MIN, CHAR_MAX);
    printf("Number of bits: %d\n", CHAR_BIT);
    printf("Size of char (bytes): %lu\n", sizeof(char));
    printf("Size of letter (bytes): %lu\n", sizeof(letter));
    return 0;
}
```

Min and max char: -128, 127
Number of bits: 8
Size of char (bytes): 1
Size of letter (bytes): 1

The `sizeof` operator gives the size of its operand in bytes

Formatted printing – printf()

- `printf(format_string, variable1, variable2, ….)`
  prints the variables according to the format string
- Declared in `<stdio.h>`

```
#include <stdio.h> /* Declaration of printf */

int main ()
{
    /* code in the next slides goes here */
    return 0;
}
```

printf for char

```
char    aCharacter ;
char    aNotherCharacter = 'X';
char    nullCharacter    = '\0';
char    ch               = '';
printf("aNotherCharacter = %c\n", aNotherCharacter);
```

```
aNotherCharacter = X
```

```
printf("nullCharacter = *%c*\n", nullCharacter);
```

```
nullCharacter = **
```

```
printf("character ch is *%c*\n", ch);
```

```
character ch is **
```

```
printf("aCharacter = *%c*\n", aCharacter);
```

```
Result is undefined
```

printf for int

```
int k = 43;
long int n = 44;
printf("k = |%d|\n", k);
k = |43|
printf("|%4d|\n", k);
|  43|
printf("|%04d|\n", k);
|0043|
printf("|%+d|\n", k);
|+43|
printf("|%+4d|\n", k);
| +43|
printf("|%1d|\n", k);
|43|
printf("n = |%ld|\n", n);
n = |44|
```

Long ints need the “%ld” specifier

printf for floats

```
float     f = 123.456789;
double    d = 123.456789;

printf("|%f|\n", f);
|123.456787|
printf("|%f|\n", d);
|123.456789|
printf("|%10.2f|\n", d);
|    123.46|
printf("|%010.2f|\n", d);
|0000123.46|
printf("|%+010.2f|\n", d);
|+000123.46|
```

See also the “%g” specifier that adjusts format according to size of the number

printf format specifiers

University of Exeter

See, for example, the Wikipedia page:

https://en.wikipedia.org/wiki/Printf

Booleans

University of Exeter

```
#include <stdio.h>
#include <stdbool.h>

int main()
{
   bool truthy = false;

   printf("truthy = %d\n", truthy);
truthy = 0
   truthy = !truthy;
   printf("truthy = %d\n", truthy);
truthy = 1
   return 0;
}
```

- No Boolean type before C99
- GNU extensions define a %B format specifier

- Booleans combined with !, && and ||
- ~, & and | are *bitwise* operators

Functions

Functions are defined with positional parameters and a single return value. They have their own scope and parameters are copies of the calling variable.

```
float fahrenheit(float degreesC)
{
    float degreesF = 9*degreesC/5 + 32;
    return degreesF;
}
```

Functions (procedures) that do not return a value are declared as returning **`void`**

```
void line_of_stars(int N)
{
    for (int n = 0; n < N; n++)
        printf(”*”);
    printf(”\n”);
    return;    }
```

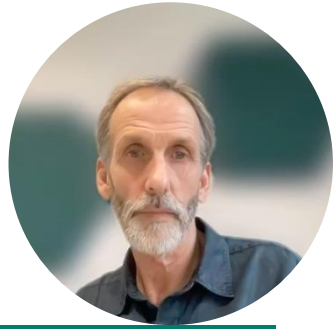
Function prototypes

Functions defined in other files are declared through their *prototypes*

```
#include <stdio.h>

float fahrenheit(float degreesC);
void line_of_stars(int N);

int main()
{
    float C = 100.0;
    float F = fahrenheit(C);
    printf(”%f Celsius is %f Fahrenheit\n”, C, F);
    line_of_stars(43);
    return 0;
}
```



```
100.000000 Celsius is 212.000000 Fahrenheit
*******************************************
```

char-int equivalence

```
char   ch = 'A';
printf("ch is %c or, equivalently, %d\n", ch, ch);
```

ch is A or, equivalently, 65

Why 65?

ch: **01000001**

Letter	ASCII Code	Binary	Letter	ASCII Code	Binary
a	097	01100001	A	065	01000001
b	098	01100010	B	066	01000010
c	099	01100011	C	067	01000011
d	100	01100100	D	068	01000100
e	101	01100101	E	069	01000101
f	102	01100110	F	070	01000110
g	103	01100111	G	071	01000111
h	104	01101000	H	072	01001000
i	105	01101001	I	073	01001001
j	106	01101010	J	074	01001010
k	107	01101011	K	075	01001011
l	108	01101100	L	076	01001100
m	109	01101101	M	077	01001101
n	110	01101110	N	078	01001110
o	111	01101111	O	079	01001111
p	112	01110000	P	080	01010000
q	113	01110001	Q	081	01010001
r	114	01110010	R	082	01010010
s	115	01110011	S	083	01010011
t	116	01110100	T	084	01010100
u	117	01110101	U	085	01010101
v	118	01110110	V	086	01010110
w	119	01110111	W	087	01010111
x	120	01111000	X	088	01011000
y	121	01111001	Y	089	01011001
z	122	01111010	Z	090	01011010

Weak typing

University of Exeter

Variable types must be declared, but can coerced to other types

```
#include <stdio.h>
int main ()
{
   unsigned char         ch        = 'A';                                  01000001
   unsigned short int    elephants = 66;                         00000000 01000010
   unsigned int          penguins  = 65603;   … 00000001 00000000 01000011

   printf("ch is %c or, equivalently, %d\n", ch, ch);
   ch = elephants;
   printf("ch is now %c or %d\n", ch, ch);
   ch = penguins;  /* Loss of precision! */
   printf("ch is now %c or %d\n", ch, ch);
   return 0;
}
```

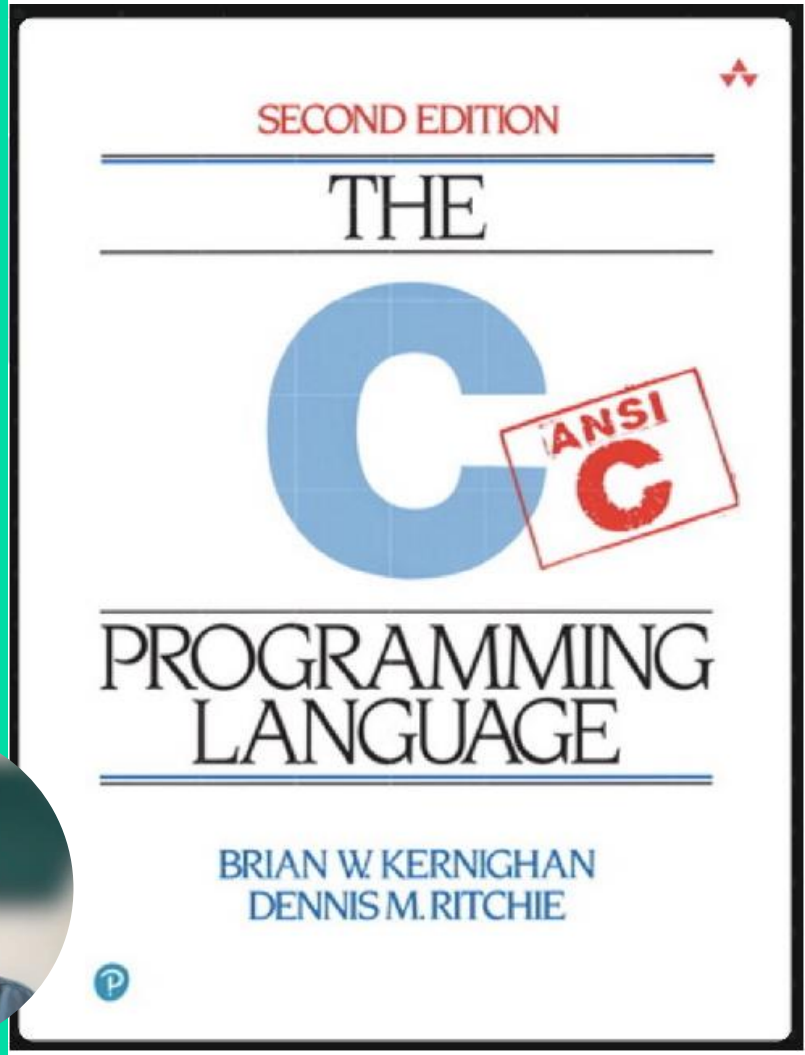
```
ch is A or, equivalently, 65
ch is now B or 66
ch is now C or 67
```

Summary

C is:

- Statically typed – types are checked before runtime
- Weakly typed – supports implicit type conversions

- Auto variables are allocated on the stack

- Control structures familiar from Java

- `printf()` for printing